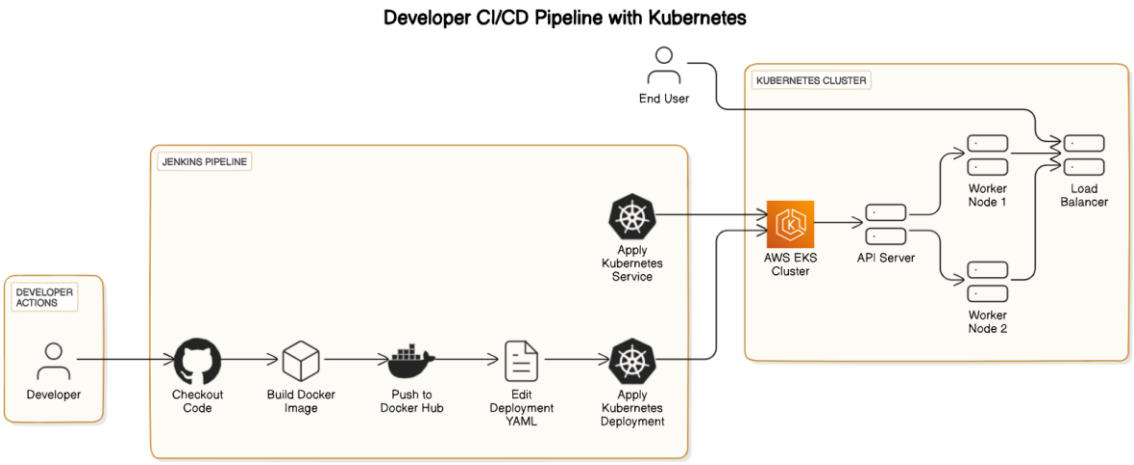
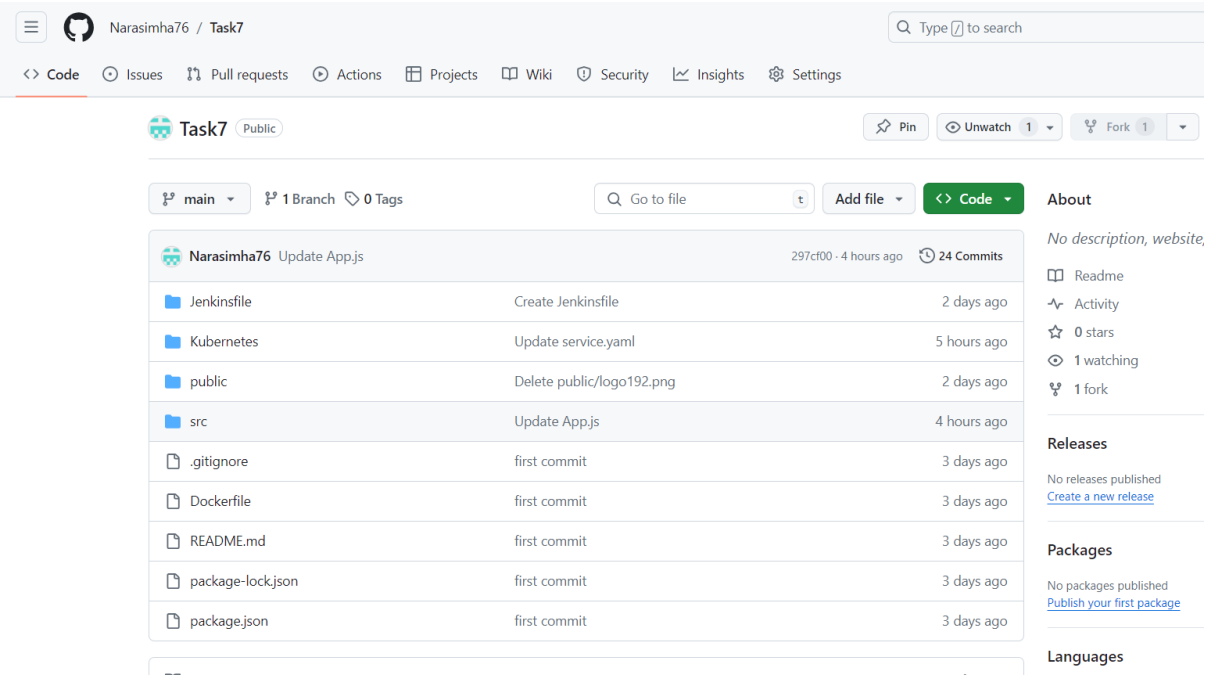
CI/CD Pipeline with Jenkins for Kubernetes Deployment



Step 1: Create a Git Repository for Storing the Source Code, Dockerfile, and Kubernetes Manifest File

Create a New Repository choose repository visibility (public/private).

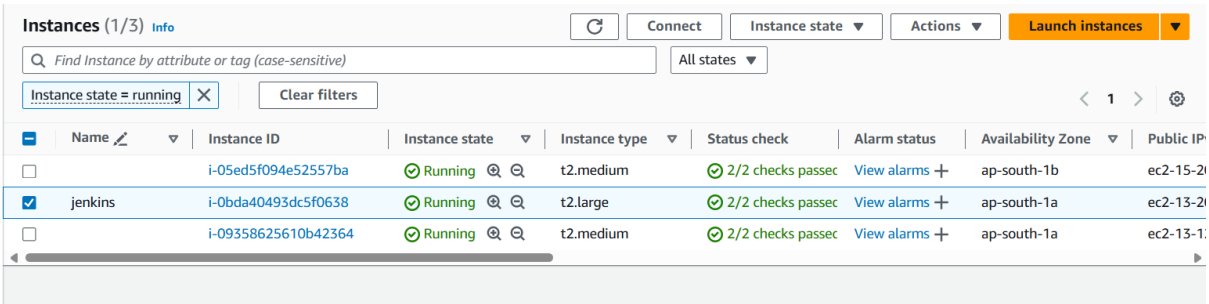
Adding the Files by Clone the repository locally, add your source code, Dockerfile, and Kubernetes manifest files, commit the changes, and push them to GitHub.



Step 2: Create an EC2 Instance and Install Required Tools

1. Create EC2 Instance:

- Launch an EC2 instance with the Ubuntu image and instance type t2.large.
- Configure instance details as needed and launch the instance.



2. Install Jenkins and Docker:

- Connect to the EC2 instance via SSH.
- Install Jenkins on the instance.
- Install Docker on the instance and configure it to work with Jenkins.

Connect to your instance using this command:

ssh -i "jenkins.pem.pem" ec2-user@ec2-<ip-address>.<region>.compute.amazonaws.com or use EC2 instance connect .

Install Jenkins on the instance using these commands:

sudo apt update

sudo apt install openjdk-11-jdk -y

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins -y

sudo systemctl status Jenkins

Install Docker on the instance using these commands:

sudo apt update

sudo apt install docker.io -y

sudo systemctl enable --now docker

3. Install kubectl and AWS CLI:

- Install kubectl to manage Kubernetes clusters.
- Install the AWS CLI to integrate the EKS cluster with the EC2 instance.

Download the latest release of kubectl and insatll AWS CLI using “sudo apt install awscli –y” and Configure AWS CLI with your access and secret keys using the command "aws configure"

Verify the installation:

```
[ec2-user@ip-172-31-34-83 ~]$ ls
Task3  Task7  jenkins  jenkins.war  kubectl.sha256
[ec2-user@ip-172-31-34-83 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-34-83 ~]$ kubectl version
Client Version: v1.30.3
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
Server Version: v1.30.2-eks-db838b0
[ec2-user@ip-172-31-34-83 ~]$ aws --version
aws-cli/2.17.21 Python/3.11.9 Linux/6.1.97-104.177.amzn2023.x86_64 exe/x86_64.amzn.2023
[ec2-user@ip-172-31-34-83 ~]$ aws configure
AWS Access Key ID [****************2R3K]:
AWS Secret Access Key [****************4CMB]:
Default region name [us-east-1]:
Default output format [json]:
[ec2-user@ip-172-31-34-83 ~]$
```

Step 3: Create EKS Cluster and Node Group

1. Create EKS Cluster:

- Go to the AWS Management Console and open the EKS service.
- Click on Create cluster and provide a Cluster name and Kubernetes version.
- Select the Role ARN with the AmazonEKSClusterPolicy attached.
- Configure VPC, Subnets, and Security Groups for network settings.

2. Create Node Group:

- In the EKS console, select the created cluster and navigate to Node Groups.
- Click Add node group, name the node group, and specify Instance types (e.g., t2.large).
- Assign the Node IAM role with the AmazonEKSWorkerNodePolicy, AmazonEC2ContainerRegistryReadOnly, and AmazonEKS_CNI_Policy policies.
- Configure Minimum, Maximum, and Desired node counts, and choose VPC, Subnets, and Security Groups as per your requirements.

3. Configure AWS CLI:

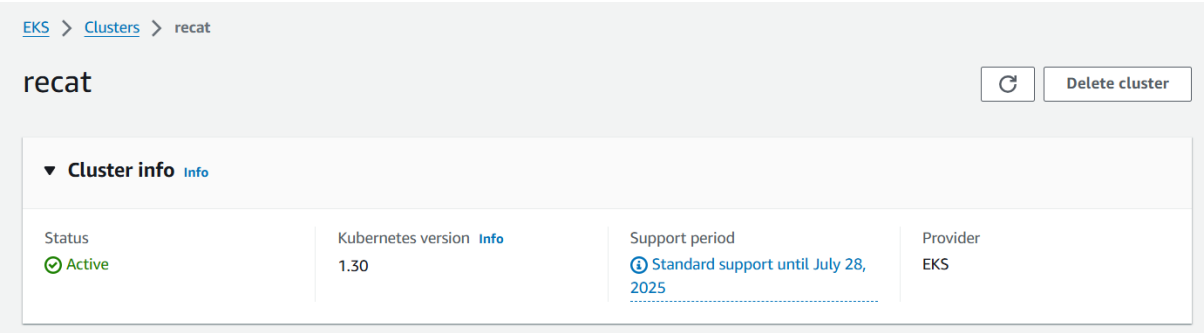
- Run aws configure on the EC2 instance to enter your AWS Access Key, Secret Access Key, Region, and Output format.

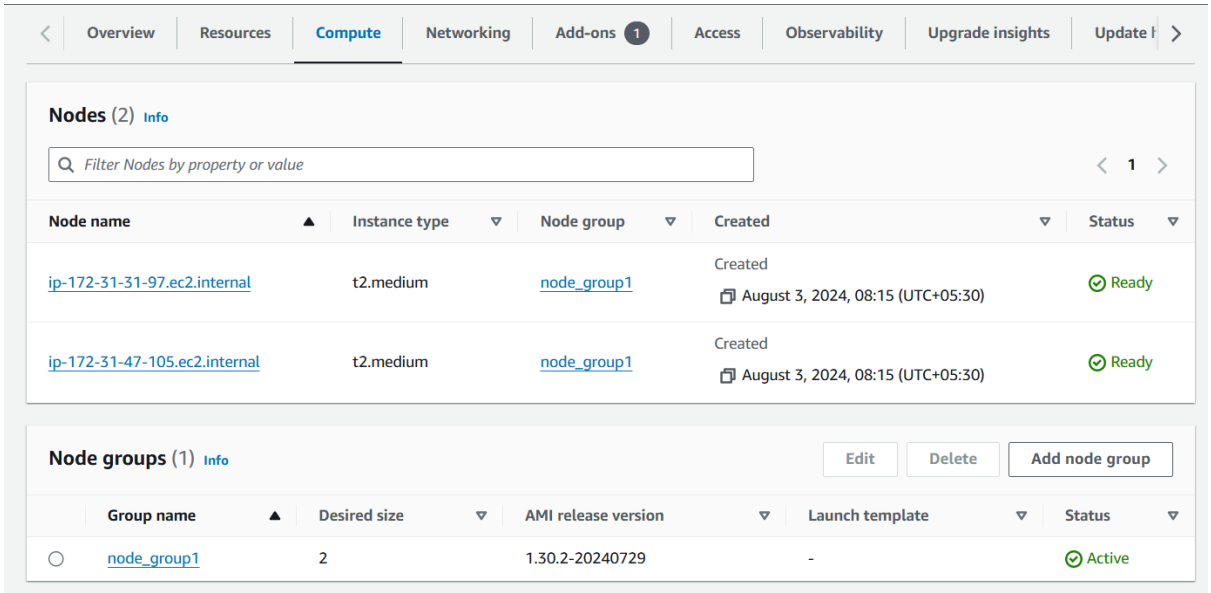
4. Update kubeconfig:

- Execute “aws eks update-kubeconfig --name <cluster-name>” to configure kubectl to use the EKS cluster.

5. Verify Node Availability:

- Use kubectl get nodes to check the status and availability of nodes in your EKS cluster.






```
[ec2-user@ip-172-31-34-83 ~]$ aws eks update-kubeconfig --name recat
Updated context arn:aws:eks:us-east-1:375388872190:cluster/recat in /home/ec2-user/.kube/config
[ec2-user@ip-172-31-34-83 ~]$ kubectl get nodes
NAME                                   STATUS   ROLES    AGE    VERSION
ip-172-31-31-97.ec2.internal           Ready    <none>   2d9h   v1.30.2-eks-1552ad0
ip-172-31-47-105.ec2.internal          Ready    <none>   2d9h   v1.30.2-eks-1552ad0
```

Step 4: Configure Jenkins

1. Start Jenkins Server:

- Run the Jenkins server with java -jar jenkins.war --httpPort=8086. Access Jenkins UI at http://<your-ec2-ip>:8086.

2. Retrieve Jenkins Admin Password:

- Get the initial admin password from the file located at /home/ec2-user/. jenkins/secrets/initialAdminPassword.

3. Install Necessary Plugins:

- Install plugins required for the pipeline: Git, Docker, AWS, Kubernetes CLI (kubectl), and Docker Hub.

4. Add Credentials:

- Add Git and Docker Hub credentials in Jenkins. Configure them under Manage Jenkins -> Manage Credentials.

5. Create Pipeline:

- Set up a new pipeline job in Jenkins and add the pipeline script to define your CI/CD process.

```
[ec2-user@ip-172-31-34-83 ~]$ ls
Task3  Task7  jenkins  jenkins.war  kubectl.sha256
[ec2-user@ip-172-31-34-83 ~]$ java -jar jenkins.war --httpPort=8086
Running from: /home/ec2-user/jenkins.war
webroot: /home/ec2-user/.jenkins/war
2024-08-05 05:09:24.725+0000 [id=1]     INFO    winstone.Logger#logInternal: Beginning extraction from war file
2024-08-05 05:09:24.818+0000 [id=1]     WARNING o.e.j.s.handler.ContextHandler#setContextPath: Empty contextPath
2024-08-05 05:09:24.886+0000 [id=1]     INFO    org.eclipse.jetty.server.Server#doStart: jetty-10.0.21; built: 2024-05-14T02:26:29.142Z; git: 12306d38ad57ba4
0dfb2b850a2900ff2de8e0357; jvm 17.0.12+7-LTS
2024-08-05 05:09:25.161+0000 [id=1]     INFO    o.e.j.w.StandardDescriptorProcessor#visitServlet: NO JSP Support for /, did not find org.eclipse.jetty.jsp.Je
ttyJspServlet
2024-08-05 05:09:25.239+0000 [id=1]     INFO    o.e.j.s.s.DefaultSessionIdManager#doStart: Session workerName=node0
2024-08-05 05:09:25.763+0000 [id=1]     INFO    hudson.WebAppMain#contextInitialized: Jenkins home directory: /home/ec2-user/.jenkins found at: $user.home/.j
enkins
2024-08-05 05:09:25.892+0000 [id=1]     INFO    o.e.j.s.handler.ContextHandler#doStart: Started w.@3277e499{Jenkins v2.470,/,file:///home/ec2-user/.jenkins/w
ar/,AVAILABLE}{/home/ec2-user/.jenkins/war}
2024-08-05 05:09:25.928+0000 [id=1]     INFO    o.e.j.server.AbstractConnector#doStart: Started ServerConnector@10b48321{HTTP/1.1, (http/1.1)}{0.0.0.0:8086}
2024-08-05 05:09:25.939+0000 [id=1]     INFO    org.eclipse.jetty.server.Server#doStart: Started Server@44a664f2{STARTING}[10.0.21,sto=0] @1739ms
2024-08-05 05:09:25.956+0000 [id=25]    INFO    winstone.Logger#logInternal: Winstone Servlet Engine running: controlPort=disabled
2024-08-05 05:09:26.214+0000 [id=31]    INFO    jenkins.InitReactorRunner$1#onAttained: Started initialization
2024-08-05 05:09:26.320+0000 [id=30]    INFO    jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2024-08-05 05:09:29.748+0000 [id=33]    INFO    jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2024-08-05 05:09:29.774+0000 [id=33]    INFO    jenkins.InitReactorRunner$1#onAttained: Started all plugins
2024-08-05 05:09:29.829+0000 [id=31]    INFO    jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2024-08-05 05:09:30.300+0000 [id=30]    INFO    h.p.b.g.GlobalTimeOutConfiguration#load: global timeout not set
2024-08-05 05:09:31.406+0000 [id=31]    INFO    jenkins.InitReactorRunner$1#onAttained: System config loaded
2024-08-05 05:09:31.409+0000 [id=31]    INFO    jenkins.InitReactorRunner$1#onAttained: System config adapted
2024-08-05 05:09:31.445+0000 [id=33]    INFO    jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2024-08-05 05:09:31.460+0000 [id=30]    INFO    jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
```

```
[ec2-user@ip-172-31-34-83 secrets]$ cat initialAdminPassword
52fff1474dbd436c875dda8f7da7bdac
[ec2-user@ip-172-31-34-83 secrets]$ pwd
/home/ec2-user/.jenkins/secrets
[ec2-user@ip-172-31-34-83 secrets]$
```

Credentials

T	P	Store ↓	Domain	ID	Name
		System	(global)	docker-hub	narasimhaswamy76/****** (This is for dockerhub)
		System	(global)	dockerhub	narasimhaswamy76/****** (this is for docker)

Dashboard > sample-react > Configuration

Configure

- General
- Advanced Project Options
- Pipeline

Advanced

Pipeline

Definition

Pipeline script

Script

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'sample-react'
        TAG = "${env.BUILD_NUMBER}"
        USER = 'narasimhaswamy76'
    }

    stages {
        stage('Checkout Code') {
            steps {
                git url: 'https://github.com/Narasimha76/Task7.git', branch: 'main'
            }
        }
        stage('Build Docker Image') {
```

Use Groovy Sandbox

Pipeline Syntax

Save Apply

Jenkins Pipeline Script Overview:

1. Checkout Code:

- Pulls the source code from the specified Git repository (https://github.com/Narasimha76/Task7.git) using the main branch.

2. Build Docker Image:

- Verifies the presence of the Dockerfile by listing files.
- Builds the Docker image with the tag my-react-app using the Dockerfile in the repository.

3. Debug:

- Provides diagnostic information about the Docker setup by displaying Docker info, version, and verifying connectivity to Docker Hub.

4. Push to Docker Hub:

- Logs into Docker Hub using credentials stored in Jenkins.
- Builds a Docker image with the tag ${IMAGE_NAME}:${TAG}, tags it, and pushes it to Docker Hub using the credentials.

5. Update Kubernetes Deployment:

- Updates the Kubernetes manifest to use the new Docker image by modifying the image tag in the react-app-deployment.yaml.
- Applies the updated deployment and service configuration to the Kubernetes cluster to deploy the new image.

Code: https://github.com/Narasimha76/Task7/blob/main/Jenkinsfile/Jenkinsfile

Step 5: Set Up the Kubernetes Manifest Files

1. Create Deployment Configuration:

- Define a Kubernetes deployment named react-app to manage the deployment of a React application.
- Set the deployment to use two replicas of the container as per your requirements, which runs the React app on port 3000.
- Specify the Docker image to be used for the deployment, which should be tagged appropriately.

2. Create Service Configuration:

- Define a Kubernetes service named react-app-service with a LoadBalancer type.
- Configure the service to expose port 80 and route traffic to port 3000 of the deployed containers.

3. Apply the Configurations:

- Use kubectl apply -f to apply the deployment and service configuration files to the Kubernetes cluster, which will set up the deployment and expose the application through the LoadBalancer.

4. Commands to apply the configurations:

- kubectl apply -f deployment.yaml
- kubectl apply -f service.yaml

YAML FILES: https://github.com/Narasimha76/Task7/tree/main/Kubernetes

Step 6: Integrate Git with Jenkins Pipeline

1. Create a GitHub Webhook:

- Navigate to your GitHub repository settings.
- Add a new webhook with the Jenkins URL: http://<your-jenkins-ip>:8080/github-webhook/.
- Set the webhook trigger to activate on push events and test the webhook to ensure it's working.

2. Generate GitHub Access Token:

- In GitHub, create a new personal access token with the required permissions for repository access.
- Add this token to Jenkins credentials by using your GitHub username and the generated token.

3. Configure Jenkins Pipeline:

- In the Jenkins pipeline configuration, enable the GitHub hook trigger option.
- Ensure that the pipeline is configured to poll SCM for changes, which will trigger the pipeline on each commit.

4. Verify Integration:

- Confirm that the Jenkins pipeline triggers automatically on commits to the GitHub repository by making a test commit and observing the pipeline execution.

By this the configuration the integration between the Github and Jenkins is created.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security and analysis

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Webhooks / Manage webhook

Settings | Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, `x-www-form-urlencoded`, *etc*). More information can be found in our developer documentation.

Payload URL *

http://13.201.41.114:8086/github-webhook/

Content type *

application/x-www-form-urlencoded

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification Disable (not recommended)

Which events would you like to trigger this webhook?

Just the `push` event.

Send me **everything**.

Let me select individual events.

Active

We will deliver event details when this hook is triggered.

Update webhook Delete webhook

Webhooks

Add webhook

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.

http://13.201.41.114:8086/github-w... *(push)*

Last delivery was successful.

Edit Delete

Dashboard > sample-react > Configuration

Configure

General

Advanced Project Options

Pipeline

Throttle builds ?

Build Triggers

Build after other projects are built ?

Build periodically ?

Generic Webhook Trigger ?

GitHub hook trigger for GITScm polling ?

Poll SCM ?

Quiet period ?

Trigger builds remotely (e.g., from scripts) ?

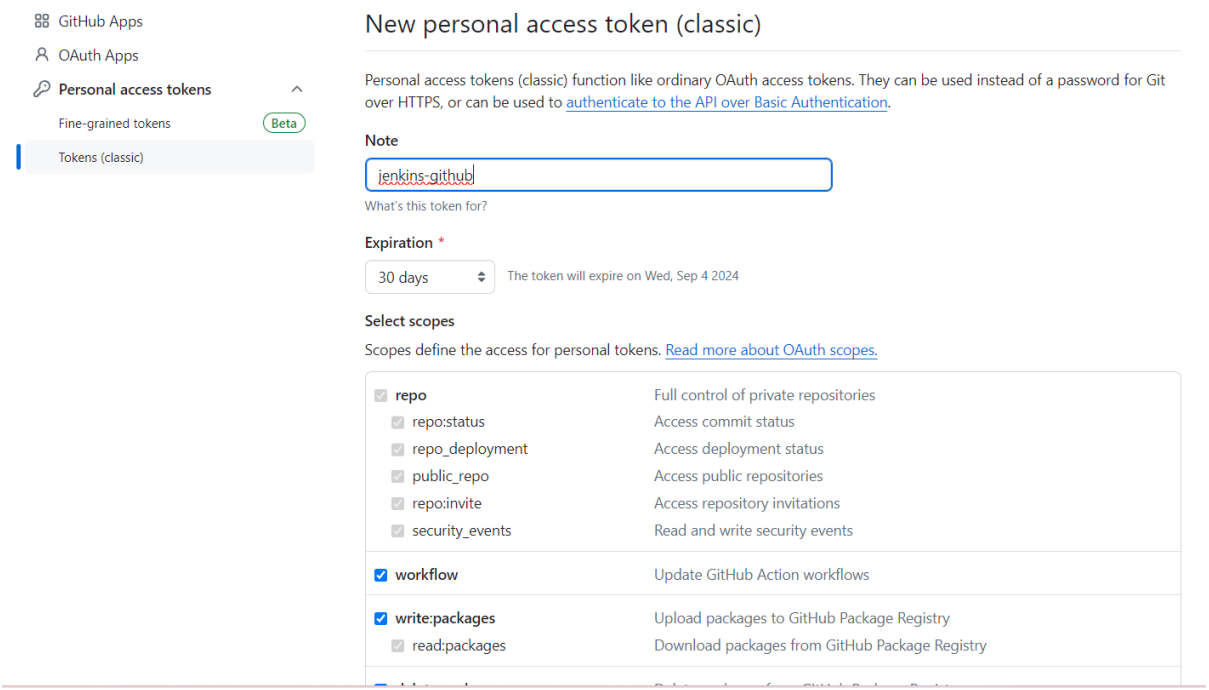
Advanced Project Options

Advanced

Pipeline

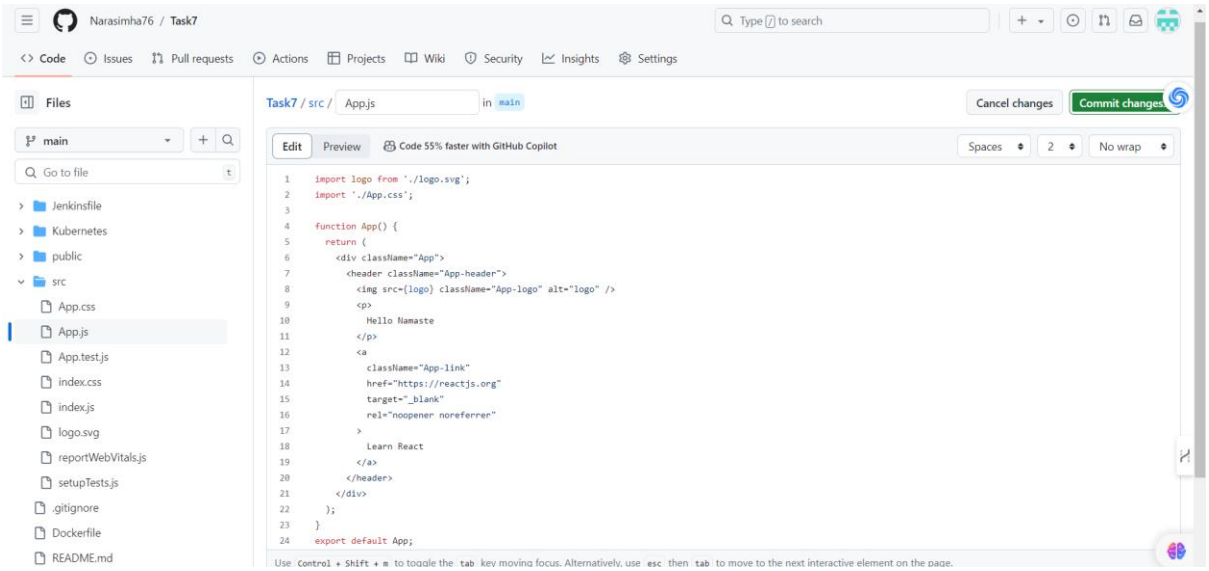
Definition

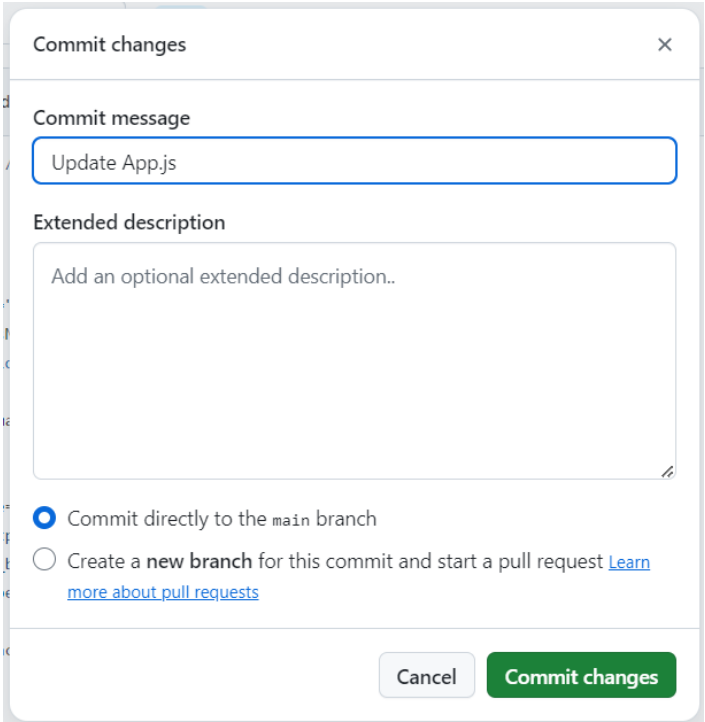
Save Apply



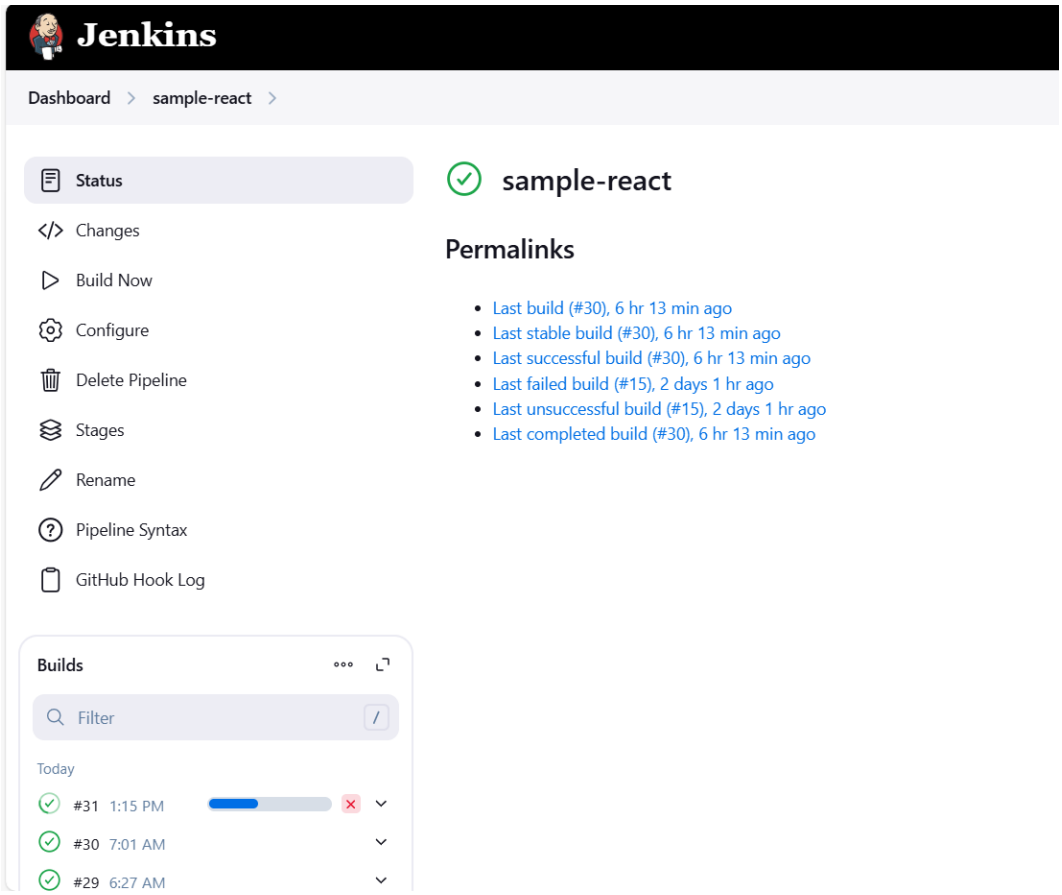
Test the Entire Flow:

1. Commit Changes: Make a commit to the Git repository to trigger the Jenkins pipeline.





2. Verify Pipeline Execution: Check the Jenkins console output to ensure the pipeline is running and review the logs for any issues.



We can also check the logs:

Dashboard > sample-react > #31

```
Use "kubectl <command> --help" for more information about a given command.
Use "kubectl options" for a list of global command-line options (applies to all commands).
[Pipeline] sh
+ cd Kubernetes
[Pipeline] sh
+ ls
Dockerfile
Jenkinsfile
Kubernetes
README.md
package-lock.json
package.json
public
src
[Pipeline] sh
+ sed -i 's|narasimhaswamy76/sample-react:.*|narasimhaswamy76/sample-react:31|' Kubernetes/react-app-deployment.yaml
+ kubectl apply -f Kubernetes/react-app-deployment.yaml
deployment.apps/react-app configured
+ kubectl apply -f Kubernetes/service.yaml
service/react-app-service unchanged
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
✕		react	N/A	2 days 2 hr #1	31 ms
✓		sample-react	5 min 12 sec #31	2 days 1 hr #15	46 sec

3. Confirm Image Deployment: Verify that the new Docker image has been pushed to Docker Hub by inspecting the Docker Hub repository.

docker hub Explore Repositories Organizations Search Docker

narasimhaswamy76 / Repositories / sample-react / General

General Tags Builds Collaborators Webhooks Settings

narasimhaswamy76/sample-react

Updated 4 minutes ago

This repository does not have a description INCOMPLETE

This repository does not have a category INCOMPLETE

Tags

This repository contains 25+ tag(s).

Tag	OS	Type	Pulled	Pushed
31		Image	4 minutes ago	4 minutes ago
30		Image	6 hours ago	6 hours ago
29		Image	7 hours ago	7 hours ago
28		Image	7 hours ago	7 hours ago

4. Check Kubernetes Deployment: Review the Jenkins workspace YAML files and use Kubernetes commands to confirm the new pod with the image and service are deployed.

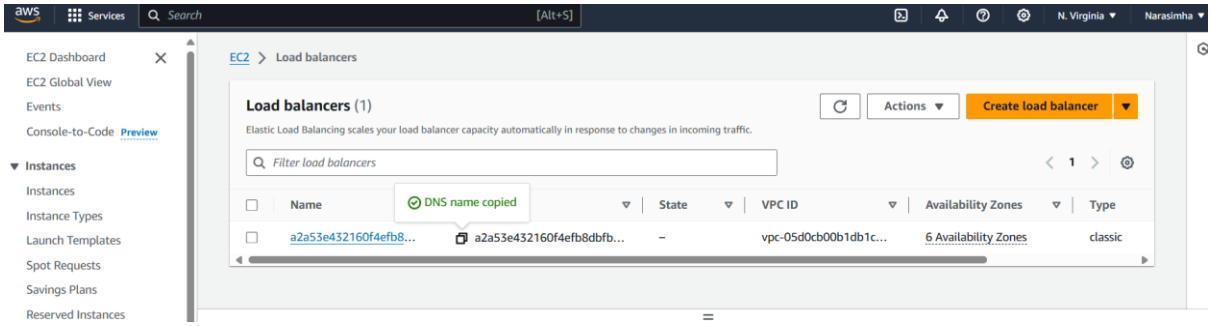
Check the image tag in the docker hub and in the deployment yaml file it should be same tag then we can ensure that the recent image is deployed in my case it will be same as 31 in both the docker hub and the deployment file.

```
[ec2-user@ip-172-31-34-83 Kubernetes]$ pwd
/home/ec2-user/.jenkins/workspace/sample-react/Kubernetes
[ec2-user@ip-172-31-34-83 Kubernetes]$ ls
react-app-deployment.yaml  service.yaml
[ec2-user@ip-172-31-34-83 Kubernetes]$ cat react-app-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: react-app
  labels:
    app: react-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: react-app
  template:
    metadata:
      labels:
        app: react-app
    spec:
      containers:
      - name: react-app
        image: narasimhaswamy76/sample-react:31
        ports:
        - containerPort: 3000
[ec2-user@ip-172-31-34-83 Kubernetes]$
```

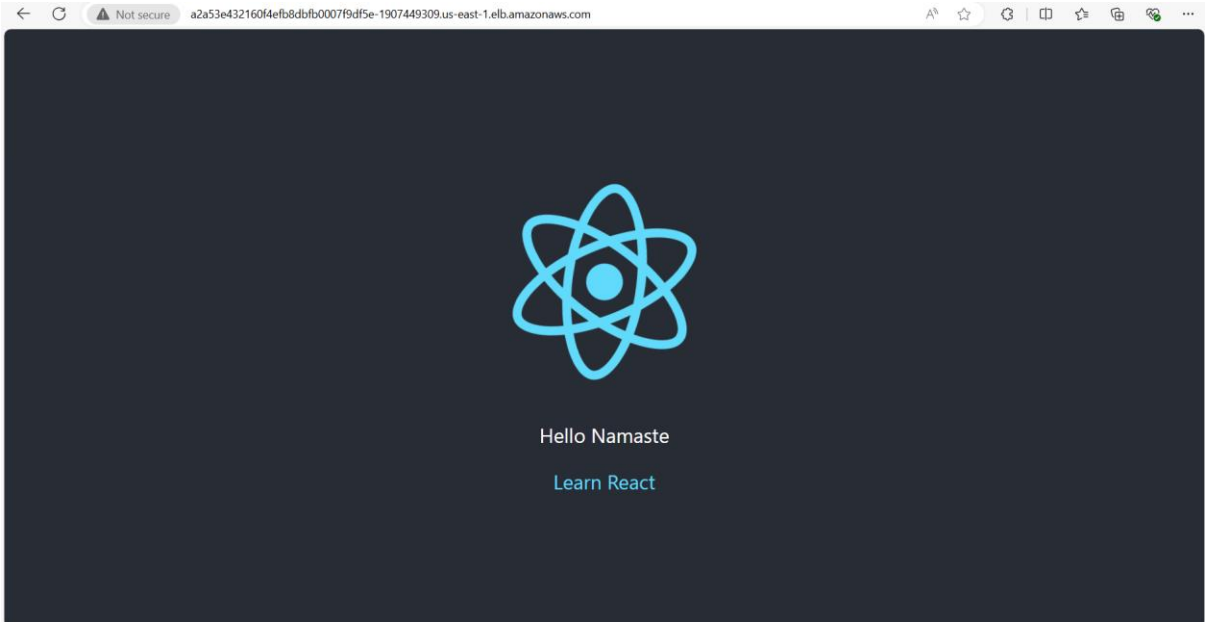
```
[ec2-user@ip-172-31-34-83 Kubernetes]$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: react-app-service
spec:
  type: LoadBalancer
  selector:
    app: react-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
[ec2-user@ip-172-31-34-83 Kubernetes]$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
react-app-b467cd55-brr9r      1/1     Running   0          10m
react-app-b467cd55-zpf49      1/1     Running   0          10m
[ec2-user@ip-172-31-34-83 Kubernetes]$ kubectl get svc
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP                                                                PORT(S)        AGE
kubernetes          ClusterIP      10.100.0.1       <none>                                                                     443/TCP        2d10h
react-app-service   LoadBalancer   10.100.117.186   a2a53e432160f4efb8dbfb0007f9df5e-1907449309.us-east-1.elb.amazonaws.com   80:30930/TCP   2d3h
[ec2-user@ip-172-31-34-83 Kubernetes]$
```

5. Validate Application: Access the Load Balancer URL in a browser to ensure the new application image is correctly deployed and functioning.

Check in the load balancer in the AWS console weather load balancer is created or not.



Copy the DNS name and paste in browser:



Ensure that your latest image is created or not in my case In Github I commited the change as Hello Namaste as same change is deployed in the kubernetes.